CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Section 3: Claude Code CLI, Configuration & Automated Workflows

Exam objectives covered

1. Configure Claude Code CLI invocations for automated CI/CD pipelines using non-interactive flags, permission modes, and cost and turn limits that prevent runaway executions.
2. Structure iterative refinement workflows by providing concrete input-output examples, targeted feedback on specific failures, and batched issue descriptions for consolidated evaluation.
3. Design Claude Code review configurations that load the correct project standards, restrict unnecessary tool access, and produce structured output suitable for automated downstream processing.
4. Apply the `context: fork` frontmatter option to Skill and slash command configurations that should execute in an isolated subagent context, preventing cross-contamination of session state.
5. Improve automated test generation quality by providing existing test files as context, defining fixture conventions, and specifying criteria that distinguish meaningful behavioral tests from trivial assertions.
6. Select the correct Claude Code configuration mechanism—CLAUDE.md, `.claude/rules/` with glob patterns, Skills, hooks, or settings permissions—based on guidance type and when it should apply.
7. Apply systematic codebase exploration strategies using Grep, Glob, and Read tools that build incremental understanding while managing context window constraints.
8. Select the appropriate Claude Code built-in tool—Grep, Glob, Read, or Bash—based on the nature of the search or file operation required for a given codebase task.

Q1

A team wants to run Claude Code inside a CI pipeline that must never pause for prompts and must stop before consuming excessive tokens or looping through too many iterations. Which configuration is most appropriate?

A. Run interactively with broad permissions so the model can decide when to stop B. Use non-interactive execution, an explicit permission mode, and max cost/turn limits C. Allow all tools and omit limits so the pipeline can finish complex work D. Use a local IDE session instead of CLI automation

Answer: B **Explanation:** CI/CD workflows should be deterministic and non-blocking, so non-interactive execution plus explicit permission controls are essential. Max cost and turn limits reduce runaway behavior. A and C are risky because they can stall or overspend, while D does not address automation requirements.

Q2

You are refining a prompt for automated bug fixing. The first run produces a patch that fixes one bug but breaks a validation rule. What feedback is most effective for the next iteration?

A. "Try harder and make it better." B. "The result is bad. Rewrite everything." C. "You fixed issue A, but regression remains: inputs over 255 chars now bypass validation. Preserve the original length check and rerun only against that failure." D. "Use your best judgment and improve quality."

Answer: C **Explanation:** Targeted feedback tied to a specific failure helps the model focus on the real defect and avoid undoing correct work. The other choices are vague and force the model to infer what went wrong, which weakens iterative refinement.

Q3

Why is it often better to batch multiple related issues into one consolidated evaluation prompt during review?

A. It guarantees each issue is fixed in a separate commit B. It helps the model compare tradeoffs across the issue set and respond in a structured, unified way C. It reduces the need for any project context D. It prevents the need for permissions configuration

Answer: B **Explanation:** Batched issue descriptions support holistic reasoning and make it easier to request a single structured output for downstream automation. A is unrelated, C is false because context still matters, and D confuses review structure with tool/permission policy.

Q4

A review workflow must apply the project's Python style guide only to `backend/**/*.py` and the React conventions only to `frontend/**/*.tsx`. Which Claude Code mechanism is the best fit?

A. A single global CLAUDE.md with all rules mixed together B. `.claude/rules/` files with appropriate glob patterns C. A hook that runs on every command and prints both guides D. A permission setting that allows Read and Grep

Answer: B **Explanation:** `.claude/rules/` with glob patterns is designed for scoped guidance that applies only to matching files. A global CLAUDE.md is too broad, hooks are for automation side effects rather than contextual guidance, and permission settings do not express coding standards.

Q5

A team wants review output that another script can parse reliably. Which instruction is best?

A. "Write a natural, free-form review in whatever style seems best." B. "Return findings as strict structured output with stable fields such as severity, file, line, summary, and recommendation." C. "Return a short poem summarizing problems." D. "Only tell us whether the code feels good."

Answer: B **Explanation:** Automated downstream processing requires predictable structure and stable fields. A, C, and D produce ambiguous or inconsistent output that is difficult to validate or transform programmatically.

Q6

When should you add `context: fork` frontmatter to a Skill or slash command?

A. When the workflow should run inside the current conversation state and reuse all prior context B. When the workflow should execute in an isolated subagent context to avoid contaminating the main session C. Only when the command needs Bash access D. Only for long outputs that exceed the context window

Answer: B **Explanation:** `context: fork` is for isolation: the task runs in a separate context so temporary reasoning, instructions, or state do not pollute the parent session. A is the opposite of the goal, while C and D confuse isolation with tool choice or output length.

Q7

Which scenario is the strongest candidate for `context: fork`?

A. A tiny in-session clarification like renaming one variable B. A reusable review command that loads project standards and performs a multi-step audit before returning a summary C. A one-line grep request for a single symbol D. A simple request to read a known file

Answer: B **Explanation:** Multi-step reusable workflows often benefit from isolated execution so their internal instructions and exploration do not bleed into the main session. A, C, and D are lightweight enough to run directly without extra isolation.

Q8

An automated test-generation prompt keeps producing shallow tests like "returns true when true is passed." Which addition is most likely to improve results?

A. Tell the model to "write more tests" B. Provide existing test files, fixture conventions, and explicit criteria favoring behavioral edge cases over trivial assertions C. Remove all project examples so the model stays unbiased D. Ask for the shortest possible test file

Answer: B **Explanation:** Existing tests and fixture patterns teach the local testing style, while explicit quality criteria steer the model toward meaningful behavior verification. A is too vague, C removes useful context, and D encourages superficial coverage.

Q9

Which instruction best distinguishes meaningful behavioral tests from trivial assertions?

A. "Increase line coverage no matter what." B. "Prefer tests that verify observable behavior, edge cases, failure modes, and contract boundaries; avoid asserting implementation trivia with no user-visible value." C. "Use one assertion per test because it looks cleaner." D. "Mock every dependency whether needed or not."

Answer: B **Explanation:** Good test-generation guidance emphasizes behavior and contract validation instead of shallow checks. A optimizes the wrong metric, C is an overgeneralized style rule, and D can create brittle, low-value tests.

Q10

Which Claude Code configuration mechanism is the best choice for durable, repository-wide guidance that contributors should see frequently?

A. CLAUDE.md B. A one-off slash command C. A transient interactive message D. A max-cost setting

Answer: A **Explanation:** CLAUDE.md is the right place for persistent, broad repository guidance. Slash commands are task-oriented, transient messages are not durable, and cost settings control execution behavior rather than coding guidance.

Q11

You need to enforce that a notification script runs automatically after a certain workflow step completes. Which mechanism best fits?

A. CLAUDE.md B. `.claude/rules/` C. Hooks D. Glob

Answer: C **Explanation:** Hooks are designed for automatic behavior tied to events or workflow steps. CLAUDE.md and rules provide guidance, not execution triggers, and Glob is just a file-matching tool.

Q12

A project wants to prevent Claude Code from using unnecessary tools during automated review runs. What is the best approach?

A. Load the correct review standards and restrict tool access through settings permissions B. Give the model every tool so it can choose freely C. Disable all instructions and rely only on the base model D. Replace project rules with a larger context window

Answer: A **Explanation:** Least-privilege tool access reduces risk and keeps reviews focused, while project standards ensure the right criteria are applied. B is overbroad, C loses important local guidance, and D does not solve permissions control.

Q13

You need to understand an unfamiliar codebase without overwhelming the context window. Which exploration strategy is best?

A. Read the entire repository file-by-file from the start B. Use Grep or Glob to narrow candidates, then Read selected files incrementally as hypotheses form C. Start with Bash-based recursive dumps of every file into one output D. Avoid reading files until after code changes are made

Answer: B **Explanation:** Incremental narrowing is the key to efficient exploration and context management. A and C flood the context window, while D delays understanding until too late.

Q14

Which built-in tool is the best first choice when you know the kind of filename you want, but not its exact location?

A. Read B. Glob C. Bash D. Grep

Answer: B **Explanation:** Glob is optimized for finding files by path or naming pattern. Read requires a known file, Grep is better for content search, and Bash is less targeted for this task.

Q15

Which built-in tool is usually the best choice when you need to search file contents for a symbol, error string, or code pattern across the repository?

A. Grep B. Glob C. Read D. Bash

Answer: A **Explanation:** Grep is purpose-built for fast repository-wide content search. Glob only matches paths, Read is for examining specific files, and Bash is more general but less direct for targeted code search.

Q16

You already know the exact file path and need to inspect a specific implementation section. Which tool is most appropriate?

A. Glob B. Grep C. Read D. Bash

Answer: C **Explanation:** Read is the best fit when the file is already known and you want to inspect its contents directly. Glob and Grep are discovery tools, while Bash is unnecessarily broad for a simple file read.

Q17

When is Bash the most appropriate built-in tool among Grep, Glob, Read, and Bash?

A. When matching filenames by wildcard B. When locating a text string across source files C. When opening a known file for inspection D. When you need a general shell operation or command execution beyond file matching/reading

Answer: D **Explanation:** Bash is the right choice for general shell tasks, command execution, or operations outside the narrower scope of the other tools. A, B, and C each map more directly to Glob, Grep, and Read respectively.

Q18

A team is choosing between CLAUDE.md, `.claude/rules/`, Skills, hooks, and settings permissions. Which mapping is most correct?

A. CLAUDE.md for durable broad guidance; rules for scoped file-pattern guidance; Skills for reusable task workflows; hooks for automatic event-driven actions; settings permissions for tool-access control B. CLAUDE.md for permissions; hooks for coding standards; Skills for glob matching; rules for cost limits; settings for test fixtures C. Skills for all permanent repository policy; hooks for all

human-readable guidance; settings for slash command content D. Rules for interactive turn limits; CLAUDE.md for tool sandboxing; hooks for context forking

Answer: A **Explanation:** This option correctly aligns each mechanism with its intended responsibility. The other mappings confuse guidance, automation, permissions, and workflow reuse, leading to brittle or misconfigured setups.